Terminology

Term	Description
lr, l (bit)	link register (special register)
pc, p (bit)	program counter (special register)
imm(n), i (bit)	immediate value (n bits)
condition, c (bit)	condition under which the instruction is executed. conditions are a combination of the status flags. see the chapter 'conditions'.
< >	parameters in angle brackets are optional
∈	is element of
⊆	subset or equal

SYNOPSIS *Encoding*

parameter value ranges

31/15.....0	(machine code)	**flags updated**

instruction explanation in syntax similar to C

Move

mov rd, rm *T1*

rd, rm ∈ { r0..r11 }

010001	10	d	mmmm	ddd

rd = rm;

movs rd, rm *T2*

rd, rm ∈ { r0..r7 }

000	00	00000	mmm	ddd

N Z

rd = rm;

movs rd, #imm8 *T1*

rd ∈ { r0..r7 }

imm8 ∈ { 0..7 }

001	00	ddd	iiiiiiii

N Z

rd = imm8;

Add

add rd, rm *T2*

rd, rm { r0..r11 }

010001	00	d	mmmm	ddd

rd = rd + rm;

adds rd, <rn>, rm *T1*

rd, rn, rm ∈ { r0..r7 }

000	11	0	0	mmm	nnn	ddd

N Z C V

rd = rn + rm; *or* rd += rm;

adcs rd, rm *T1*

rd, rm ∈ { r0..r7 }

010000	0101	mmm	ddd

N Z C V

rd = rd + rm + carry;

adds rd, rn, #imm3 *T1*

rd, rn ∈ { r0..r7 }

imm3 ∈ { 0..7 }

000	11	1	0	iii	nnn	ddd

N Z C V

rd = rn + imm3;

adds rd, #imm3 *T1*

rd ∈ { r0..r7 }

imm8 ∈ { 0..255 }

001	10	ddd	iiiiiiii

N Z C V

rd = rd + imm8;

Subtract

subs rd, <rn>, rm *T1*

rd, rn, rm { r0..r7 }

000	11	0	1	mmm	nnn	ddd

N Z C V

rd = rn - rm; *or* rd -= rm;

rsbs rd, <rm> *T1*

rd, rm { r0..r7 }

010000	1001	nnn	ddd

N Z C V

rd = 0 - rn; *or* rd = 0 - rd;

sbcs rd, rm *T1*

rd, rm { r0..r7 }

010000	0110	mmm	ddd

N Z C V

rd = rd - rm - (-carry);

subs rd, rn, #imm3 *T1*

rd, rn ∈ { r0..r7 }

imm3 ∈ { 0..7 }

000	11	1	1	iii	nnn	ddd

N Z C V

rd = rn - imm3; *or* rd -= imm3;

subs rd, #imm8 *T2*

rd ∈ { r0..r7 }

imm8 ∈ { 0..255 }

001	11	ddd	iiiiiiii

N Z C V

rd = rd - imm8;

Multiply

muls rd, rn *T1*
rd, rn ∈ { r0..r7 }

010000	1101	nnn	ddd

N Z

```
rd = rd * rn;
```

Compare

cmp rn, rm *T1*
rn, rm ∈ { r0..r7 }

010000	1010	mmm	nnn

N Z C V

```
update_flags(rn - rm);
```

cmp rn, rm *T2*
rn, rm ∈ { r0..r11 }

010001	01	n	mmmm	nnn

N Z C V

```
update_flags(rn - rm);
```

cmn rn, rm *T1*
rn, rm ∈ { r0..r7 }

010000	1011	mmm	nnn

N Z C V

```
update_flags(rn + rm);
```

cmp rn, #imm8 *T1*
rn ∈ { r0..r7 }
imm8 ∈ { 0..255 }

001	01	nnn	iiiiiiii

N Z C V

```
update_flags(rn - imm8);
```

Logical

ands rd, rm *T1*
rd, rm ∈ { r0..r7 }

010000	0000	mmm	ddd

N Z

```
rd = rd & rm;
```

orrs rd, rm *T1*
rd, rm ∈ { r0..r7 }

010000	1100	mmm	ddd

N Z

```
rd = rd | rm;
```

eors rd, rm *T1*
rd, rm ∈ { r0..r7 }

010000	0001	mmm	ddd

N Z

```
rd = rd ^ rm;
```

bics rd, rm *T1*
rd, rm ∈ { r0..r7 }

010000	1110	mmm	ddd

N Z

```
rd = rd & ~rm;
```

mvns rd, rm *T1*
rd, rm ∈ { r0..r7 }

010000	1111	mmm	ddd

N Z

```
rd = ~rm;
```

tst rn, rm *T1*
rn, rm ∈ { r0..r7 }

010000	1000	mmm	nnn

N Z

```
update_flags(rn & rm);
```

Shift / Rotate

lsls rd, rm *T1*
rd, rm ∈ { r0..r7 }

010000	0010	mmm	ddd

N Z C

```
rd = rd << rm;
```

lsrs rd, rm *T1*
rd, rm ∈ { r0..r7 }

010000	0011	mmm	ddd

N Z C

```
rd = rd >> rm;
```

asrs rd, rm *T1*
rd, rm ∈ { r0..r7 }

010000	0100	mmm	ddd

N Z C

```
rd = extend_sign(rd >> rm);
```
(**rd** = **rd** / $2^{\mathbf{rm}}$)

lsls rd, <rm>, #imm5 *T1*
rd, rm ∈ { r0..r7 }
imm5 ∈ { 0..31 }

000	00	iiiii	mmm	ddd

N Z C

```
rd = rm << imm5; or rd <<= imm5;
```

lsrs rd, <rm>, #imm5 *t1*
rd, rm ∈ { r0..r7 }
imm5 ∈ { 0..31 }

000	01	iiiii	mmm	ddd

N Z C

```
rd = rm >> imm5; or rd >>= imm5;
```

asrs rd, <rm>, #imm5 *T1*
rd, rm ∈ { r0..r7 }
imm5 ∈ { 0..31 }

000	10	iiiii	mmm	ddd

N Z C

```
rd = extend_sign(rm >> imm5);
or rd = extend_sign(rd >> imm5);
```
(**rd** = **rm** / $2^{\mathbf{imm5}}$)

rors rd, rm *T1*
rd, rm ∈ { r0..r7 }

010000	0111	mmm	ddd

N Z C

```
rd = rd >> rm | rd << (32 - rm);
(rotate right)
```

Load

ldr rt, [rn, rm] *T1*
rt ∈ { r0..r7, pc, sp }
rn, rm ∈ { r0..r7 }

0101	100	mmm	nnn	ttt

```
rt = *((uint32_t)(rn + rm));
```

ldrh rt, [rn, rm] *T1*
rt, rn, rm ∈ { r0..r7 }

0101	101	mmm	nnn	ttt

```
rt = *((uint16_t)(rn + rm));
```

ldrb rt, [rn, rm] *T1*
rt, rn, rm ∈ { r0..r7 }

0101	110	mmm	nnn	ttt

```
rt = *((uint8_t)(rn + rm));
```

ldr rt, [rn, <#imm5>] *T1*
rt ∈ { r0..r7, pc, sp }
rn ∈ { r0..r7 }
imm5 ∈ { 0..31 } << 2

01101	iiiii	nnn	ttt

```
rt = *((uint32_t)(rn + imm5 << 2));
```

ldrh rt, [rn, <#imm5>] *T1*
rt, rn ∈ { r0..r7 }
imm5 ∈ { 0..31 } << 1

1000	1	iiiii	nnn	ttt

```
rt = *((uint16_t)(rn + imm5 << 1));
```

ldrb rt, [rn, <#imm5>] *T1*
rt, rn ∈ { r0..r7 }
imm5 ∈ { 0..31 }

011	1	1	iiiii	nnn	ttt

```
rt = *((uint8_t)(rn + imm5));
```

ldr rt, [#imm8] *T1*
rt ∈ { r0..r7, pc, sp }
imm8 ∈ { 0..255 } << 2

01001	ttt	iiiiiiii

```
rd = *((uint32_t)(pc + imm8 << 2));
```

ldrsh rt, [rn, rm] *T1*
rt, rn, rm ∈ { r0..r7 }

0101	111	mmm	nnn	ttt

```
rt = extend_sign(
((uint16_t)(rn + rm)) );
```

ldrsb rt, [rn, rm] *T1*
rt, rn, rm ∈ { r0..r7 }

0101	011	mmm	nnn	ttt

```
rt = extend_sign(
((uint8_t)(rn + rm)) );
```

Store

str rt, [rn, rm] *T1*
rt, rn, rm ∈ { r0..r7 }

0101	000	mmm	nnn	ttt

```
*((uint32_t)(rn + rm)) = rt;
```

strh rt, [rn, rm] *T1*
rt, rn, rm ∈ { r0..r7 }

0101	001	mmm	nnn	ttt

```
*((uint16_t)(rn + rm)) = rt;
```

strb rt, [rn, rm] *T1*
rt, rn, rm ∈ { r0..r7 }

0101	010	mmm	nnn	ttt

```
*((uint8_t)(rn + rm)) = rt;
```

str rt, [rn, <#imm5>] *T1*
rt, rn ∈ { r0..r7 }
imm5 ∈ { 0..31 } << 2

01100	iiiii	nnn	ttt

```
*((uint32_t)(rn + imm5 << 2)) = rt;
```

strh rt, [rn, <#imm5>] *T1*
rt, rn ∈ { r0..r7 }
imm5 ∈ { 0..31 } << 1

1000	0	iiiii	nnn	ttt

```
*((uint16_t)(rn + imm5 << 1)) = rt;
```

strb rt, [rn, <#imm5>] *T1*
rt, rn ∈ { r0..r7 }
imm5 ∈ { 0..31 }

011	1	0	iiiii	nnn	ttt

```
*((uint8_t)(rn + imm5)) = rt;
```

Load / Store Multiple Registers

ldm rn<!>, { registers } *T1*
rn ∈ { r0..r7 }
registers ⊆ { r0..r7 }
<!> also load rn

1100	1	nnn	rrrrrrrr

```
for (k = 0; k < #regs; k++){
regs[k] = *((uint32_t)(rn + k));
}
```

stm rn<!>, { registers } *T1*
rd, ∈ { r0..r7 }
registers ⊆ { r0..r7 }
<!> also load rn

1100	0	nnn	rrrrrrrr

```
for (k = 0; k < #regs; k++){
((uint32_t)(rn + k)) = regs[k];
}
```

Push / Pop

push { registers } *T1*
registers ⊆ { r0..r7, lr }

1011	0	10	l	rrrrrrrr

```
for (k = 0; k < #regs; k++){
stack[k] = regs[k];
}
```

pop { registers } *T1*
registers ⊆ { r0..r7, pc }

1011	1	10	p	rrrrrrrr

```
for (k = 0; k < #regs; k++){
regs[k] = stack[k];
}
```

Branch

b<condition> label *T1*
imm8 ∈ { 0..255 } << 1
(= distance to label)

1101	cccc	iiiiiiii

```
if (condition) pc += imm8 << 1;
```

b label *T2*
imm11 ∈ { 0..2047 } << 1
(= distance to label)

11100	iiiiiiiiiii

```
pc = pc + imm11 << 1;
```

bx rm *T1*
rm ∈ { r0..r11 }

010001	11	0	mmmm	000

```
pc = rm & ~1u;
```

blx rm *T1*
rm ∈ { r0..r11 }

010001	11	1	mmmm	000

```
lr = pc | 1u;
pc = rm & ~1u;
```

bl label *T1*
label (imm23) ∈ { -8'388'608..8'388'607 } << 1

11110	S	iiiiiiiiiii	11	J	1	K	iiiiiiiiiii

I1 = not(J xor S) I2 = not(K xor S)
imm23 = s:i1:i2:imm10:imm11:0

```
lr = pc | 1u;
pc = pc + imm23 << 1;
```

Stack Operations

ldr rt, [sp, <#imm8>] *T2*
rt ∈ { r0..r7 }
imm8 ∈ { 0..255 } << 2

1001	1	ttt	iiiiiiii

```
rt = *((uint32_t*)(sp + imm8 << 2));
```

str rt, [sp, <#imm8>] *T2*
rt ∈ { r0..r7 }
imm8 ∈ { 0..255 } << 2

1001	0	ttt	iiiiiiii

```
((uint32_t*)(sp + imm8 << 2)) = rt;
```

add rd, sp, #imm8 *T1*
rd ∈ { r0..r7 }
imm8 ∈ { 0..255 } << 2

1010	1	ddd	iiiiiiii

```
rd = sp + imm8 << 2;
```

add sp, #imm7 *T2*
imm7 ∈ { 0..127 } << 2

1011	0000	0	iiiiiii

```
sp = sp + imm7 << 2;
```

sub sp, #imm7 *T1*
imm7 ∈ { 0..127 } << 2

1011	0000	1	iiiiiii

```
sp = sp - imm7 << 2;
```

Extend

sxth rd, rm *T1*
rd, rm ∈ { r0..r7 }

1011	0010	00	mmm	ddd

```
rd = extend_sign((uint16_t)rm);
```

sxtb rd, rm *T1*
rd, rm ∈ { r0..r7 }

1011	0010	01	mmm	ddd

```
rd = extend_sign((uint8_t)rm);
```

uxth rd, rm *T1*
rd, rm ∈ { r0..r7 }

1011	0010	10	mmm	ddd

```
rd = extend_zeroes((uint16_t)rm);
```

uxtb rd, rm *T1*
rd, rm ∈ { r0..r7 }

1011	0010	11	mmm	ddd

```
rd = extend_zeroes((uint8_t)rm);
```

Status Register

mrs rd, apsr *T1*
rd ∈ { r0..r11 }

11110	0	1111	1	0	1111	10	0	0	dddd	00000000

```
rd = apsr;
```

*'**mrs**' can move other special registers, but we use only **apsr**. This would change the red coloured bits.*

msr apsr, rn *T1*
rn ∈ { r0..r11 }

11110	0	1110	0	0	nnnn	10	0	0	10	0	0	00000000

```
apsr = rn;
```

*'**msr**' can move other special registers, but we use only **apsr**. This would change the red coloured bits.*

Conditions

This table lists the conditions to be used with conditional instructions. In CT1 we use only one instruction with a condition, the branch with label T1 encoded.
Note that the conditions have distinct meaning, depending on the operands being unsigned or signed.

bits	symbol	flags	context	description
0000	eq	Z == 1	all	equal
0001	ne	Z == 0	all	not equal
0010	cs / hs	C == 1	unsigned	carry set / higher than or same
0011	cc / lo	C == 0	unsigned	carry cleared / lower than
0100	mi	N == 1	signed	minus / negative
0101	pl	N == 0	signed	plus / positive or zero
0110	vs	V == 1	signed	overflow set (overflow)
0111	vc	V == 0	signed	overflow cleared (no overflow)
1000	hi	C == 1 && Z == 0	unsigned	higher than
1001	ls	C == 0 || Z == 1	unsigned	lower than or same
1010	ge	N == V	signed	greater than or equal
1011	lt	N != V	signed	less than
1100	gt	Z == 0 && N == V	signed	greater than
1101	le	Z == 1 || N != V	signed	less than or equal
1110	al		all	always == no condition
1111	-			-

Flag Dependent

symbol	description	flags
eq	equal	Z == 1
ne	not equal	Z == 0
cs	carry set	C == 1
cc	carry cleared	C == 0
mi	minus / negative	N == 1
pl	plus / positive or zero	N == 0
vs	overflow set (overflow)	V == 1
vc	overflow cleared (no overflow)	V == 0

Arithmetic Unsigned

symbol	description	flags
eq	equal	Z == 1
ne	not equal	Z == 0
hs (cs)	unsigned higher than or same	C == 1
lo (cc)	unsigned lower than	C == 0
hi	higher than	C == 1 && Z == 0
ls	lower than or same	C == 0 || Z == 1

Arithmetic Signed

symbol	description	flags
eq	equal	Z == 1
ne	not equal	Z == 0
mi	minus / negative	N == 1
pl	plus / positive or zero	N == 0
vs	overflow set (overflow)	V == 1
vc	overflow cleared (no overflow)	V == 0
ge	signed greater than or equal	N == V
lt	signed less than	N != V
gt	signed greater than	Z == 0 && N == V
le	signed less than or equal	Z == 1 || N != V

Disassembly Table

```
[ 0000 / 0x0 ]
0000 0000 00mm mddd     movs rd, rm           // rd = rm (alias for `lsls rd, rm, #0`)
0000 0iii iimm mddd     lsls rd, rm, #imm5    // rd = rm << imm5
0000 1iii iimm mddd     lsrs rd, rm, #imm5    // rd = rm >> imm5

[ 0001 / 0x1 ]
0001 0iii iimm mddd     asrs rd, rm, #imm5    // rd = rm << imm5 (keep sign)
0001 100m mmnn nddd     adds rd, rn, rm       // rd = rn + rm
0001 101m mmnn nddd     subs rd, rn, rm       // rd = rn - rm
0001 110i iinn nddd     adds rd, rn, #imm5    // rd = rn + imm5
0001 111i iinn nddd     subs rd, rn, #imm5    // rd = rn - imm5

[ 0010 / 0x2 ]
0010 0ddd iiii iiii     movs rd, #imm8        // rd = #imm8
0010 1nnn iiii iiii     cmp  rn, #imm8        // flags = rn - imm8

[ 0011 / 0x3 ]
0011 0ddd iiii iiii     adds rd, #imm8        // rd = rd + imm8
0011 1ddd iiii iiii     subs rd, #imm8        // rd = rd - imm8

[ 0100 / 0x4 ]
0100 0000 00mm mddd     ands rd, rm           // rd = rd & rm
0100 0000 01mm mddd     eors rd, rm           // rd = rd ^ rm
0100 0000 10mm mddd     lsls rd, rm           // rd = rd << rm
0100 0000 11mm mddd     lsrs rd, rm           // rd = rd >> rm
0100 0001 00mm mddd     asrs rd, rm           // rd = rd >> rm (keep sign)
0100 0001 01mm mddd     adcs rd, rm           // rd = rd + rm + carry
0100 0001 10mm mddd     sbcs rd, rm           // rd = rd - rm - (-carry)
0100 0001 11mm mddd     rors rd, rm           // rd = rd << (32 - rm) | rd >> rm
0100 0010 00mm mddd     tst  rd, rm           // flags = rd & rm
0100 0010 01mm mddd     rsbs rd, rm, #0       // rd = 0 - rm (`neg` is an alias to `rsb`)
0100 0010 10mm mnnn     cmp  rn, rm           // flags = rn - rm
0100 0010 11mm mnnn     cmn  rn, rm           // flags = rn + rm
0100 0011 00mm mddd     orrs rd, rm           // rd = rd | rm
0100 0011 01mm mddd     muls rd, rm, rd       // rd = rd * rm
0100 0011 10mm mddd     bics rd, rm           // rd = rd & ~rm (bit clear)
0100 0011 11mm mddd     mvns rd, rm           // rd = ~rm
0100 0100 dmmm mddd     add  rd, rm           // rd = rd + rm
0100 0101 nmmm mnnn     cmp  rn, rm           // flags = rn - rm
0100 0110 dmmm mddd     mov  rd, rm           // rd = rm
0100 0111 0mmm m...     bx   rm               // pc = rm
0100 0111 1mmm m...     blx  rm               // lr = pc | 1u, pc = rm
0100 1ttt iiii iiii     ldr  rt, [pc, #imm11] // rt = *((uint32_t*)(pc % 4 + (imm11 << 2))
                        ldr  rt, =label       // label/literal is stored in the literal
                        ldr  rt, =literal     // pool and loaded via an offset from the PC

[ 0101 / 0x5 ]
0101 000m mmnn nttt     str   rt, [rn, rm]    // *((uint32_t*)(rn + rm)) = rt
0101 001m mmnn nttt     strh  rt, [rn, rm]    // *((uint16_t*)(rn + rm)) = rt
0101 010m mmnn nttt     strb  rt, [rn, rm]    // *((uint8_t*)(rn + rm)) = rt
0101 011m mmnn nttt     ldrsb rt, [rn, rm]    // rt = *((int8_t*)(rn + rm))
0101 100m mmnn nttt     ldr   rt, [rn, rm]    // rt = *((uint32_t*)(rn + rm))
0101 101m mmnn nttt     ldrh  rt, [rn, rm]    // rt = *((uint16_t*)(rn + rm))
0101 110m mmnn nttt     ldrb  rt, [rn, rm]    // rt = *((uint8_t*)(rn + rm))
0101 111m mmnn nttt     ldrsh rt, [rn, rm]    // rt = *((int16_t*)(rn + rm))

[ 0110 / 0x6 ]
0110 0iii iinn nttt     str rt, [rn, #imm5]   // *((uint32_t*)(rn + (imm5 << 2))) = rt
0110 1iii iinn nttt     ldr rt, [rn, #imm5]   // rt = *((uint32_t*)(rn + (imm5 << 2)))

[ 0111 / 0x7 ]
0111 0iii iinn nttt     strb rt, [rn, #imm5]  // *((uint8_t*)(rn + imm5)) = rt
0111 1iii iinn nttt     ldrb rt, [rn, #imm5]  // rt = *((uint8_t*)(rn + imm5))

[ 1000 / 0x8 ]
1000 0iii iinn nttt     strh rt, [rn, #imm5]  // *((uint16_t*)(rn + (imm5 << 1))) = rt
1000 1iii iinn nttt     ldrh rt, [rn, #imm5]  // rt = *((uint16_t*)(rn + (imm5 << 1)))
```

```
[ 1001 / 0x9 ]
1001 0ttt iiii iiii     str rt, [sp, #imm8]   // *((uint32_t*)(sp + (imm8 << 2))) = rt
1001 1ttt iiii iiii     ldr rt, [sp, #imm8]   // rt = *((uint32_t*)(sp + (imm8 << 2)))

[ 1010 / 0xA ]
1010 0ddd iiii iiii     adr rd, label         // rd = (pc % 4) + (imm8 << 2)
1010 1ddd iiii iiii     add rd, sp, #imm8     // rd = sp + (imm8 << 2)

[ 1011 / 0xB ]
1011 0000 0iii iiii     add sp, sp, #imm7     // sp = sp + (imm7 << 2)
1011 0000 1iii iiii     sub sp, sp, #imm7     // sp = sp - (imm7 << 2)
1011 00i1 iiii innn     cbz rn, label         // `armv7-m` and later only!
1011 0010 00mm mddd     sxth rd, rm           // rd = extend_sign((int16_t)rm)
1011 0010 01mm mddd     sxtb rd, rm           // rd = extend_sign((int8_t)rm)
1011 0010 10mm mddd     uxth rd, rm           // rd = (uint16_t)rm
1011 0010 11mm mddd     uxtb rd, rm           // rd = (uint8_t)rm
1011 010l rrrr rrrr     push {lr, r0..r7}     // push selected registers to the stack (l = lr)
1011 0110 0100 ....     -                     // unpredictable
1011 0110 0101 0...     setend le             // armv7-a/r only!
1011 0110 0101 1...     setend be             // armv7-a/r only!
1011 0110 0110 0aif     cpsie <i><f>          // enable processor state i=irq, f=fiq
1011 0110 0111 0aif     cpsid <i><f>          // disable processor state i=irq, f=fiq
1011 0110 011x 1...     -                     // unpredictable
1011 10i1 iiii innn     cbnz rn, label        // `armv7-m` and later only!
1011 1010 00mm mddd     rev rd, rm            // rd[31:0] = rm[7:0, 15:8, 23:16, 31:24]
1011 1010 01mm mddd     rev16 rd, rm          // rd[31:0] = rm[23:16, 31:24, 7:0, 15:8]
1011 1010 10.. ....     -                     // undefined
1011 1010 11mm mddd     revsh rd, rm          // rd[31:0] = extend_sign((int32_t)rm[7:0, 15:8])
1011 110p rrrr rrrr     pop {pc, r0..r7}      // pop selected registers from the stack (p = pc)
1011 1110 iiii iiii     bkpt #imm8            // breakpoint, arg ignored by hw
1011 1111 0000 0000     nop                   // do nothing
1011 1111 0001 0000     yield                 // signal to hw to suspend/resume threads
1011 1111 0010 0000     wfe                   // wait for event
1011 1111 0011 0000     wfi                   // wait for interrupt
1011 1111 0100 0000     sev                   // signal event to multi-processor system
1011 1111 cccc mmmm     itsel cond            // `armv7-m` and later only!

[ 1100 / 0xC ]
1100 0nnn rrrr rrrr     stmia rn<!> {reg0-7}  // store selected registers at address in `rn`
1100 1nnn rrrr rrrr     ldmia rn<!> {reg0-7}  // load selected registers from address in `rn`

[ 1101 / 0xD ]
1101 cccc iiii iiii     b<cond> label         // if (cond) goto label
1101 1110 .... ....     -                     // undefined

[ 1110 / 0xE ]
1110 0iii iiii iiii     b label               // goto label
1110 1... .... ....     -                     // `armv7-m` and later only!

[ 1111 / 0xF ]
1111 0sii iiii iiii 11j1 kiii iiii iiii     bl label      // lr = pc | 1u, goto label
1111 0011 1110 1111 1000 dddd ssss ssss     mrs rd, rs    // rd = rs (rs = special register)
1111 0011 1000 mmmm 1000 1000 ssss ssss     msr rs, rm    // rs = rm (rs = special register)
1111 0011 1011 1111 1000 1111 0100 1111     dsb           // data synchronisation barrier
1111 0011 1011 1111 1000 1111 0101 1111     dmb           // data memory barrier
1111 0011 1011 1111 1000 1111 0110 1111     isb           // instruction synchronisation barrier
```

Status Flags

The N, Z, C, and V condition flags are held in the APSR.

flag	description
N	Set to 1 when the result of the operation is negative, cleared to 0 otherwise.
Z	Set to 1 when the result of the operation is zero, cleared to 0 otherwise.
C	Set to 1 when the operation results in a carry, or when a subtraction results in no borrow, cleared to 0 otherwise.
V	Set to 1 when the operation causes overflow, cleared to 0 otherwise.

Carry is set in the following situations:

- For an addition, including the comparison instruction **CMN**, C is set to 1 if the addition produced a carry (that is, an unsigned overflow), and to 0 otherwise.
- For a subtraction, including the comparison instruction **CMP**, C is set to 0 if the subtraction produced a borrow (that is, an unsigned underflow), and to 1 otherwise.
- For non-addition/subtractions that incorporate a shift operation, C is set to the last bit shifted out of the value by the shifter.

Overflow is set when the result of a signed add, subtract, or compare is greater than or equal to 2^{31}, or less than -2^{31}. Or in other words, when the sign would be different, if the operation was executed at infinite resolution.